

Low Power Configurable Multi Clock System

UART-Controlled Register File & ALU

Design and simulation report

Ahmed Hossam eldin Osman

REF_CLK	100 MHz	Language	Verilog / SystemVerilog
UART_CLK	3.6864 MHz (115 200 baud)	Architecture	10 blocks, 2 clock domains
Verification	225 / 225 checks passed	Date	October 2, 2026

Contents

1. Introduction and specifications	3
2. System architecture	4
3. Multi-clock design: CDC, RDC, clock gating, pipelining	5
3.1 Clock and reset domains	5
3.2 Clock-domain crossing (CDC)	5
3.3 Reset-domain crossing (RDC)	6
3.4 Clock gating for power management	6
3.5 Pipelining through the stages	7
4. Module reference	7
5. Simulation and functional verification	9
5.1 Environment and results	9
5.2 SYS_CTRL functionality through the tests	9
5.3 Functional coverage (measured)	11
5.4 Command walkthroughs	11
5.5 Configuration matrix	16
5.6 Multi-clock behaviour in simulation	16
5.7 Verification traceability	18
5.8 Reproducing the simulations	19
6. Backend summary (brief)	19
7. Review notes	20

1. Introduction and specifications

System_Top is a digital system controlled entirely over a UART link. A host sends command frames; the design decodes them, reads or writes a register file, runs ALU operations, and returns the results over the same link. It is built as a **multi-clock system**: a 100 MHz reference domain holds the controller, register file and ALU, and a 3.6864 MHz UART domain holds the serial interfaces and their divided clocks. Safe communication between the domains, controlled reset release, ALU clock gating and pipelining are the main design themes; the report concentrates on how each of them behaves in simulation.

Parameter	Value
Reference clock (REF_CLK)	100 MHz (10 ns)
UART clock (UART_CLK)	3.6864 MHz = 115 200 baud × 32
Serial frame	1 start + 8 data (LSB first) + optional parity + 1 stop; 11 bit-times with parity = 95.5 μ s
Reset	external asynchronous, active low; synchronized per clock domain
Reset defaults	REG2 = 0x81 (prescale 32, parity on, even), REG3 = 0x20 (TX divide 32)
Register file / ALU / FIFO	16 × 8 bit; 16-bit ALU result, 15 functions; async FIFO 8 × 8 bit

Command set

Command	Frames (host → DUT)	Response (DUT → host)
Register write	AA, address, data	none
Register read	BB, address	register value
ALU with operands	CC, A, B, function	result[7:0], then result[15:8]
ALU, no operands	DD, function (uses REG0 = A, REG1 = B)	result[7:0], then result[15:8]

ALU functions

Code	Function	Code	Function	Code	Function
0x0	ADD	0x5	OR	0xA	A = B → 1
0x1	SUB	0x6	NAND	0xB	A > B → 2
0x2	MUL	0x7	NOR	0xC	A < B → 3
0x3	DIV (÷0 gives 0)	0x8	XOR	0xD	A » 1
0x4	AND	0x9	XNOR	0xE	A « 1 (0xF undefined → 0)

2. System architecture

The design has ten blocks in three groups (clock domain 1: REF_CLK; clock domain 2: UART_CLK; synchronizers). The block diagram from the project specification is reproduced below; the colors identify the clock each block runs on.

Final System

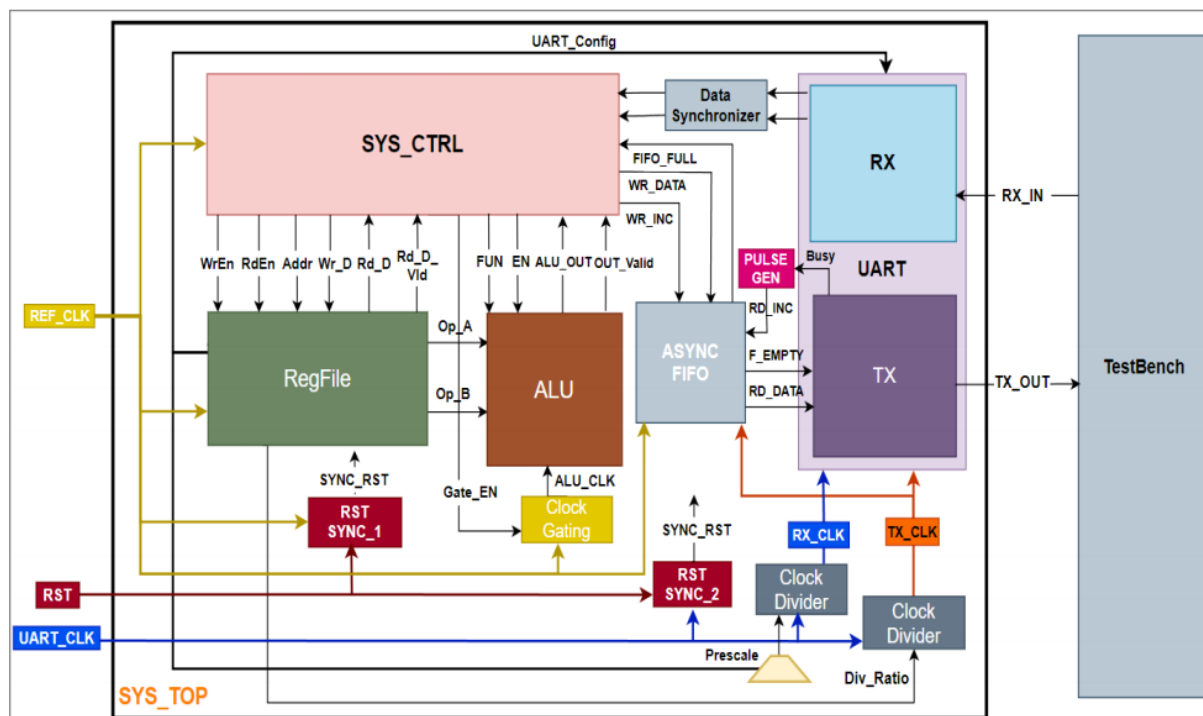


Figure 1. Final System block diagram (from the project specification). Reference clock: 100 MHz.

Group	Blocks	Role
Domain 1 (REF_CLK)	SYS_CTRL	Command FSM: decodes RX frames, drives RegFile, ALU, clock gate and the response FIFO
	RegFile, ALU, Clock Gating	16 × 8 registers (REG0/REG1 → ALU, REG2 → UART, REG3 → TX divider); 16-bit ALU with registered result; glitch-free ALU clock gate
Domain 2 (UART_CLK)	UART_RX, UART_TX, PULSE_GEN, Clock Dividers	Serial receiver (oversampling) and transmitter; busy-to-pulse for the FIFO read; TX_CLK (÷ REG3) and RX_CLK (÷ 1/2/4 by prescale)
Synchronizers	RST SYNC ×2, Data Synchronizer, ASYNC FIFO	Per-domain reset release; RX byte into REF_CLK; responses from REF_CLK to TX_CLK

Life of a command. UART_RX assembles the byte and raises RX_D_VALID for the stop-bit time → the Data Synchronizer turns it into a one-cycle REF_CLK pulse and captures the byte → SYS_CTRL decodes it and reads/writes RegFile or enables the ALU through the clock gate → results are written into the ASYNC FIFO → UART_TX sees the FIFO leave empty in its own domain, serializes the byte and pops the FIFO through PULSE_GEN.

3. Multi-clock design: CDC, RDC, clock gating, pipelining

This section explains the concepts and the technique used; the corresponding simulation evidence is in Section 5.6. Numbers labelled "measured" come from simulating the supplied RTL with REF_CLK = 100 MHz.

3.1 Clock and reset domains

Clock	Source	Frequency	Used by
REF_CLK	input	100 MHz (10 ns)	RegFile, SYS_CTRL, CLK_GATE, FIFO write side, Data Synchronizer, RST_SYNC_1
ALU_CLK	CLK_GATE	pulsed on demand	ALU
UART_CLK	input	3.6864 MHz (271.3 ns)	both clock dividers, RST_SYNC_2
TX_CLK	UART_CLK ÷ REG3	115.2 kHz at REG3 = 32 (8680.6 ns)	UART_TX, PULSE_GEN, FIFO read side
RX_CLK	UART_CLK ÷ {1, 2, 4}	3.6864 / 1.8432 / 0.9216 MHz	UART_RX

TX_CLK and RX_CLK are generated by registers clocked by UART_CLK, so they are *derived clocks of one family*; the only unrelated pair is REF_CLK and the UART family, so every data crossing is between those two.

Arithmetic: $f_{TX} = f_{UART} / \text{REG3} = 3.6864 \text{ MHz} / 32 = 115.2 \text{ kHz}$; $RX_CLK = f_{\text{baud}} \times \text{Prescale}$ (prescale 32 / 16 / 8 → $UART_CLK \div 1 / 2 / 4$), giving the receiver *Prescale* clocks per bit.

3.2 Clock-domain crossing (CDC)

Concept. A flop that samples a signal changing near its clock edge may go metastable. With resolution time t_r , $MTBF = e^{t_r/\tau} / (T_0 \cdot f_{clk} \cdot f_{data})$; a second flop adds almost a full clock period to t_r , improving MTBF by $e^{T_{clk}/\tau}$. Single bits therefore use a **2-FF synchronizer**; multi-bit values are never synchronized bit by bit but use a **synchronized enable with a held bus**, or **gray-coded pointers**.

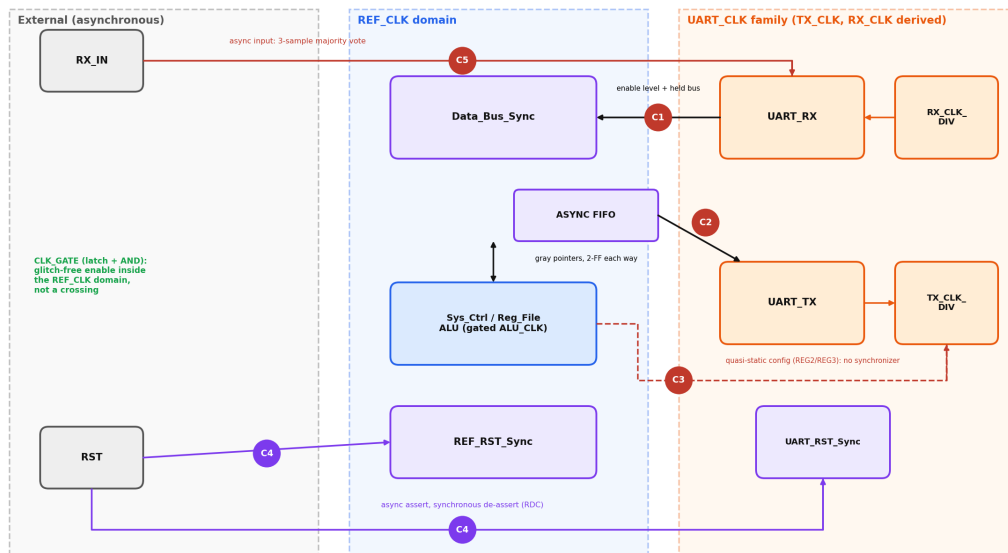


Figure 2. Clock- and reset-domain map with the five crossings analysed.

#	Crossing	Technique	Measured / note
C1	RX byte + valid: RX_CLK → REF_CLK	Data Synchronizer: 2-FF on the enable, edge-detect pulse, bus captured on the pulse. The bus is stable well before the enable (held for the stop bit, ~870 REF cycles)	pulse 21–28 ns after RX_D_VALID rises (phase dependent)
C2	Response bytes: REF_CLK → TX_CLK	Async FIFO, gray-coded pointers, 2-FF DF_SYNC each way	write → EMPTY de-assert 17.0 μs; → start bit 8.7 μs
C3	REG2 / REG3 config: REF_CLK → UART	quasi-static, no synchronizer	reconfigure only while the link is idle

#	Crossing	Technique	Measured / note
C4	RST: async → both domains	RST_SYNC per domain (RDC, Section 3.3)	release 15 ns vs 478 ns after RST
C5	RX_IN: external → RX_CLK	3-sample majority vote, no separate 2-FF stage	filters glitches

Gray code for the FIFO. Binary pointers can change many bits at once (0111 → 1000), so a synchronizer may capture a value that was never present. In gray code, $g = b \oplus (b \gg 1)$, consecutive values differ in one bit; a pointer sampled mid-change is either the old or the new value, so a flag is merely late. With 8 entries each pointer has $\log_2 8 + 1 = 4$ bits (address + wrap bit): **EMPTY** when read and (synchronized) write pointers are equal, **FULL** when they are equal except the top two bits differ.

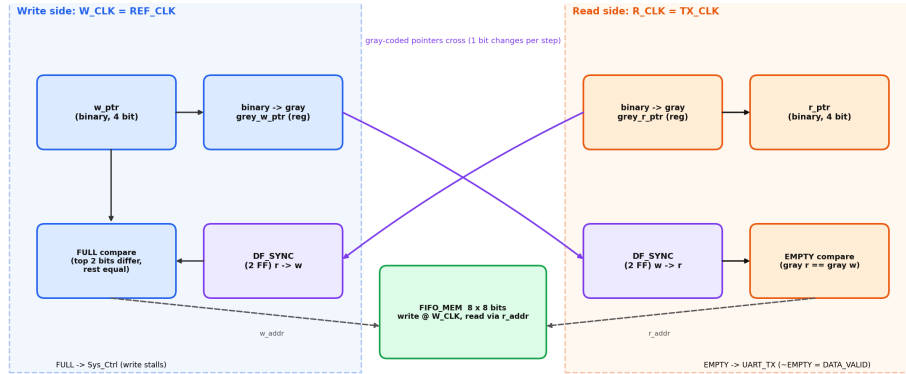


Figure 3. ASYNC_FIFO: gray-coded pointers cross through a 2-FF DF_SYNC in each direction.

Gray conversion and FULL compare (FIFO_WR.v)

```

24 grey_w_ptr <= w_ptr ^ (w_ptr >> 1);
28 assign w_full = grey_r_ptr[addr_width] != grey_w_ptr[addr_width] &&
29 grey_r_ptr[addr_width - 1] != grey_w_ptr[addr_width - 1] &&
30 grey_r_ptr[addr_width - 2 : 0] == grey_w_ptr[addr_width - 2 : 0];

```

Review note. Review notes. (C3) REG2/REG3 reach the UART family without a synchronizer, which is valid for quasi-static values, but should be declared static in constraints and kept as a host rule (no reconfiguration during a frame). (C5) RX_IN is sampled by the oversampling flops directly; adding a 2-FF stage on RX_IN would close this crossing conventionally, at the cost of one RX_CLK of latency that oversampling absorbs.

3.3 Reset-domain crossing (RDC)

Asserting reset asynchronously is harmless, but *releasing* it asynchronously can violate recovery/removal timing. RST_SYNC is therefore a 2-stage shift register cleared asynchronously by RST: the output asserts immediately and releases only after two edges of the destination clock. One instance runs on REF_CLK (SYNC_REF_RST: RegFile, SYS_CTRL, ALU, FIFO write side, Data Synchronizer) and one on UART_CLK (SYNC_UART_RST: dividers, UART, PULSE_GEN, FIFO read side).

Reset synchronizer: a 1 shifts in on every destination clock (RST_Sync.v)

```

11 sync_reg <= 'b0;
13 sync_reg[NUM_STAGES - 1 : 0] <= {1'b1, sync_reg[NUM_STAGES - 1 : 1]};
16 assign sync_RST = sync_reg[0];

```

The two releases are deliberately not simultaneous (measured: REF domain 15 ns, UART domain 478 ns after RST goes high). This is safe because: assertion is simultaneous; the **producer leaves reset last** (everything starts with a byte from UART_RX, so SYS_CTRL sees RX_D_VALID = 0 during the skew); REG2/REG3 defaults are loaded when reset asserts, so the UART domain sees valid configuration at release; and both FIFO pointers are zero before any write can occur.

3.4 Clock gating for power management

Concept. Dynamic power is $P_{\text{dyn}} = \alpha \cdot C \cdot V^2 \cdot f$. A free-running clock toggles the clock pins of a block every cycle ($\alpha = 1$) whether or not data changes; gating an idle block drives its clock activity to zero. The ALU is used for a few edges per command, so it is gated. A plain AND gate would create runt pulses if the enable changed while CLK is high, so CLK_GATE puts a **latch transparent only while CLK is low** in front of the AND: the enable can only reach the gate in the low phase, and GATED_CLK always carries whole pulses.

Latch + AND clock gate (CLK_GATE.v)

```

13 if(!CLK)      // active low
15 Latch_Out <= CLK_EN ;
20 assign GATED_CLK = CLK && Latch_Out ;

```

SYS_CTRL registers CLK_EN for one cycle in the ALU states, so the ALU receives **three** clock edges per command: **one compute edge** (ALU_EN = 1: result and OUT_VALID captured) and **two flush edges** (ALU_EN = 0: ALU_OUT and OUT_VALID return to zero, so the next ALU_FUNC state does not see a stale OUT_VALID). Measured over a 677 μ s window containing one ALU command: **3** ALU_CLK edges against **67 709** REF_CLK edges (0.0044 %). The ALU is small (16-bit result register plus valid flag), so the absolute saving is modest; the value is a correct, glitch-free structure that scales to larger blocks. After clock-tree synthesis ALU_CLK must not lead REF_CLK (the high result byte is sampled on the same edge that clears ALU_OUT).

3.5 Pipelining through the stages

$f_{\max} = 1 / (t_{\text{clk} \rightarrow q} + t_{\text{logic}} + t_{\text{setup}} + t_{\text{skew}})$: a register inside a long path splits t_{logic} across two cycles, raising $f_{\max}$ for one extra cycle of latency, and a register at every block boundary lets each block be timed independently.

Stage	Registers inserted	Effect
UART_RX	3 sample flops → majority vote → deserializer; error flags and DATA_VALID registered	short paths at the oversampling clock
Data Synchronizer	2-FF enable, edge register, pulse register, 8-bit capture	3–4 cycle pipeline that is also the CDC structure
SYS_CTRL	state and every output (WrEn, RdEn, address, data, ALU_EN/FUN, CLK_EN, FIFO write) registered	no decode logic on block outputs
RegFile / ALU	registered read data + valid; ALU: combinational compute then result register	slow multiply/divide bounded flop-to-flop
ASYNC FIFO / UART_TX	registered gray pointers, 2-FF DF_SYNC; TX registers the byte and parity config on accept	decouples 100 MHz producer from 115.2 kHz consumer

Because RX and TX are independent state machines on separate clocks and the FIFO buffers up to eight bytes, SYS_CTRL returns to fetch right after queuing a response and can accept the next command while the transmitter is still sending. Measured core latency (RX_D_VALID → first FIFO write): **61–68 ns** across register read and ALU operations, i.e. 6–7 REF_CLK cycles including the CDC pipeline, or about **0.07 %** of one frame; system throughput is set by the serial link, not the core (Section 5.6).

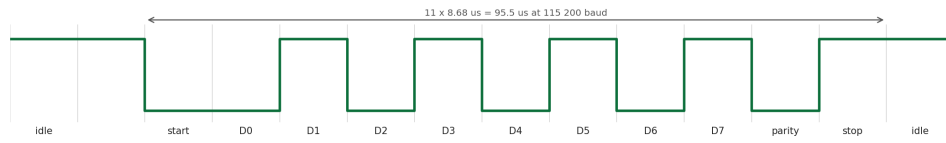
4. Module reference

A summary of every block with the design decisions that matter for verification. Code is limited to short highlights; the clock-crossing blocks are covered in Section 3.

Module	Function	Key design points
Reg_File	16 × 8 registers, synchronous write, registered read	Read returns data one cycle after RdEn with RdData_Valid. REG0/REG1 = ALU operands, REG2 = UART config {prescale[5:0], PAR_TYP, PAR_EN}, REG3 = TX divide ratio. Reset: REG2 = 0x81, REG3 = 0x20, others 0.
ALU	16-bit result, 15 functions	Combinational compute, registered result + valid on the gated clock; Enable = 0 forces result and valid to 0. ÷0 → 0; compares return 1/2/3; inverting functions are evaluated in 16-bit context (upper byte 0xFF); code 0xF → 0.
SYS_CTRL	10-state command FSM	Decodes AA/BB/CC/DD (other bytes ignored), one state per argument frame, all outputs registered; writes responses LSB first into the FIFO and honours FIFO_FULL (Section 5.2).
CLK_GATE	ALU clock gate	Low-transparent latch + AND (Section 3.4).
Clock dividers	TX_CLK, RX_CLK	Counter toggles the output every N/2 input cycles (50 % duty); ratio 0/1 bypasses. RX ratio from prescale: 32 → 1, 16 → 2, 8 → 4.
UART_TX	serializer	Registers byte + parity config on accept; IDLE → Start → Transmission → (PARITY) → Stop; busy high outside IDLE; parity = XOR of data bits (inverted for odd).
UART_RX	oversampling receiver	3 samples at P/2–1, P/2, P/2+1 with majority vote; start/parity/stop checkers; DATA_VALID in STOP only if no error flag; back-to-back frames supported.

Module	Function	Key design points
PULSE_GEN	level → pulse	Rising-edge detector on TX busy → FIFO read increment (byte already registered by TX).
Synchronizers	RST_SYNC, Data Sync, ASYNC FIFO	See Section 3.

Frame for byte 0xAA, even parity (LSB first): 1 start + 8 data + 1 parity + 1 stop = 11 bit-times



One bit cell, Prescale P = 8 shown (RX_CLK cycles 0 .. P-1). Samples taken at P/2-1, P/2, P/2+1 → majority vote

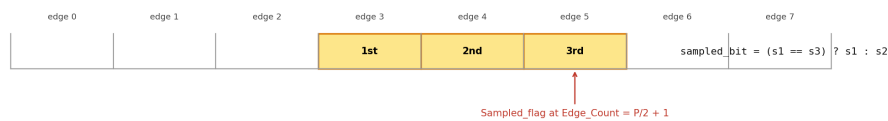


Figure 4. Frame format (0xAA, even parity) and the three-sample majority vote in the middle of each bit (RX).

Why oversample at the bit centre. The receiver finds the start edge, counts P clocks per bit and samples around P/2. The first sample after the start edge is P/2 clocks from either bit edge, so accumulated timing error over the following ten bits must stay below about half a bit: a few percent baud mismatch is tolerated at best.

Majority vote (Sampler.v)

```

38 if(first_sample == third_sample)
39   sampled_bit = first_sample;
41   sampled_bit = second_sample;

```

ALU: registered result (excerpt) (ALU.v)

```

31 if(B == 0)
37   Comb_OUT = A / B;
101 ALU_OUT <= 'b0;
102 OUT_VALID <= 'b0;
106 ALU_OUT <= Comb_OUT;
107 OUT_VALID <= OUT_VALID_Comb;

```

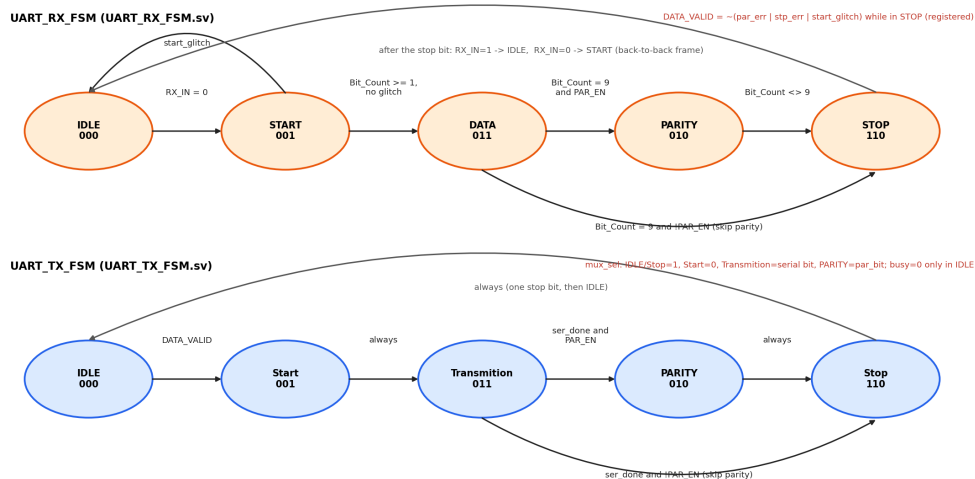


Figure 5. UART_RX and UART_TX state machines (from the .sv FSMs).

5. Simulation and functional verification

5.1 Environment and results

The system testbench acts as the UART master: it frames bytes on RX_IN (parity per REG2), decodes the response frames on TX_OUT and compares them with expected values. REF_CLK is 100 MHz and UART_CLK 3.6864 MHz (115 200 baud). For each prescale (32, 16, 8) it writes REG2/REG3 through the UART, then runs the checks below and prints PASS/FAIL per check and a summary. The waveforms in this section were captured in the simulator; coverage numbers in 5.3 were measured by running the supplied testbench source (+QUICK) with a monitor attached to SYS_CTRL.

Check (per prescale)	Count
RX_CLK and TX_CLK period (divider outputs)	2
Read-back of REG2, REG3 (configuration applied)	2
Register writes produce no response (no unexpected TX traffic)	1
Register read-back at 0x04, 0x07, 0x0B, 0x0F	4
ALU with operands (CC): 25 operations × 2 response bytes	50
REG0/REG1 preload: no response	1
ALU without operands (DD): 5 operations × 2 bytes	10
Mixed CC/DD re-check, end-of-sweep TX idle	5
Total per prescale (× 3 prescales = 225)	75

Measured. Supplied log: **PASSED: 225 FAILED: 0.** (The log prints every ALU operation with the label "ADD"; the returned values show it is a sweep across functions, decoded in Section 5.4.) REG2 reads back 0x81, 0x41, 0x21 (prescale 32/16/8), and the RX_CLK periods (271.3, 542.5, 1085.1 ns) and TX_CLK period (8680.6 ns) match the clock arithmetic of Section 3.1.

Testbench log (first checks of the prescale-32 pass; time in ps)

```
[703805000] PASS  RX_CLK period 271.3 ns (Prescale 32 -> UART_CLK/1)
[712485000] PASS  TX_CLK period 8680.6 ns (= 1 bit @ 115200 baud)
[1024985000] PASS  RD REG2 (UART cfg) : 0x81
[1337485000] PASS  RD REG3 (div ratio) : 0x20
[2804499000] PASS  RF writes: no response : no unexpected TX traffic
[3116999000] PASS  RD RF[0x04] : 0x85
[3429499000] PASS  RD RF[0x07] : 0x1c
[3741999000] PASS  RD RF[0x0b] : 0xff
[4054499000] PASS  RD RF[0x0f] : 0x21
[4575333000] PASS  CC ADD  A=0xc8 B=0x07 LSB : 0xcf
[4679499000] PASS  CC ADD  A=0xc8 B=0x07 MSB : 0x00
[5200333000] PASS  CC ADD  A=0x62 B=0xec LSB : 0x4e
```

5.2 SYS_CTRL functionality through the tests

SYS_CTRL is the only block that sees every command, so each testbench phase exercises a specific FSM path, and the response checks validate its outputs (address, data, FIFO push order, enables). The state codes below were observed in the waveforms (Section 5.4).

Sys_Ctrl command FSM (from Sys_Ctrl.v) - unknown command bytes leave the FSM in fetch

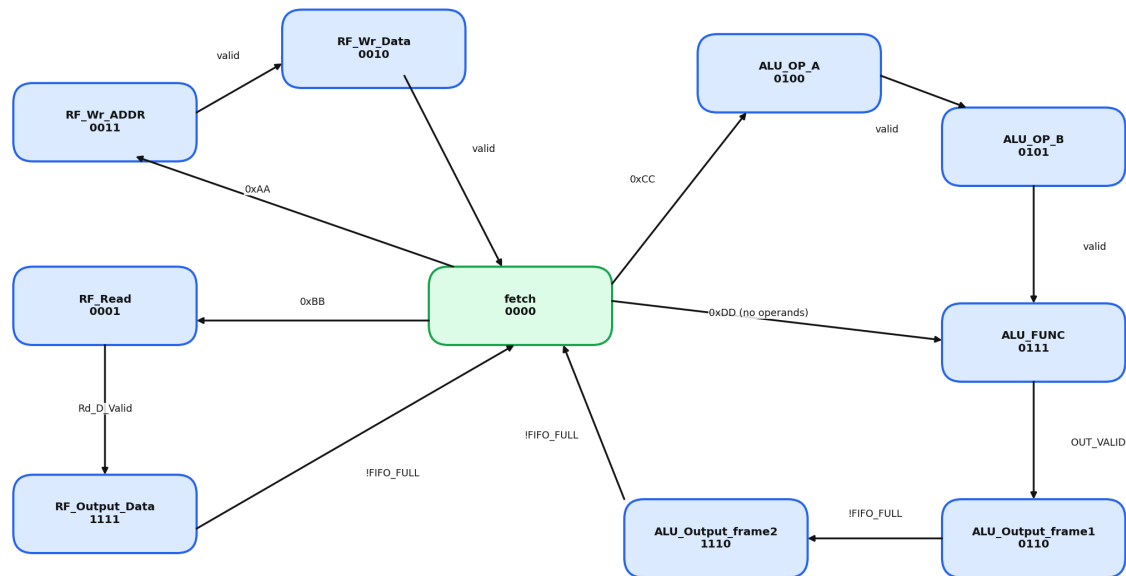


Figure 6. SYS_CTRL state diagram (4-bit state codes as in the RTL and waveforms).

SYS_CTRL function	FSM path	Test stimulus	What the test proves
Register write	0 → 3 → 2 → 0	start-up writes to REG2/REG3; writes to 0x04, 0x07, 0x0B, 0x0F; REG0/REG1 preload	address then data latched in order; no response is pushed (TX idle check)
Register read	0 → 1 → F → 0	read-back of REG2/REG3 and the four general registers	RdEn/address timing, RdData pushed to the FIFO, byte returned intact
ALU with operands	0 → 4 → 5 → 7 → 6 → E → 0	25 CC operations per prescale incl. corners (0xFF × 0xFF, A × 0, ÷, compares, shifts)	REG0/REG1 loaded, ALU_EN/CLK_EN, OUT_VALID handshake, LSB before MSB
ALU without operands	0 → 7 → 6 → E → 0	5 DD operations on REG0 = 0x7A, REG1 = 0x0C (ADD, SUB, AND, SHL, GT)	existing operands reused; shorter path returns to fetch
Sequencing	return to fetch each time	mixed CC then DD with persisting operands	no state is left behind between commands
Reconfiguration	write path to REG2	prescale 32 → 16 → 8 mid-run	new RX_CLK ratio takes effect; later frames still decode

Where SYS_CTRL pulses its enables (registered outputs) (Sys_Ctrl.v)

```

173 RdEn    <= 1'b1;
180 FIFO_W_INC <= 1'b1;
193 WrEn    <= 1'b1;
203 WrEn    <= 1'b1;
213 WrEn    <= 1'b1;
221 CLK_EN <= 1'b1;
223 ALU_EN <= 1'b1;
228 CLK_EN <= 1'b1;
231 FIFO_W_INC <= 1'b1;
236 CLK_EN <= 1'b1;
239 FIFO_W_INC <= 1'b1;

```

SYS_CTRL output	Asserted	Seen in simulation
WrEn	one cycle after the data frame of a write	write waveform (strobe with address 3)
RdEn	one cycle after the address frame of a read	read waveform
ALU_EN + CLK_EN	one cycle in state 7 (function byte)	zoom marker 3: compute edge on ALU_CLK
CLK_EN only	one cycle in states 6 and E	zoom marker 5: flush edges

SYS_CTRL output	Asserted	Seen in simulation
FIFO_WR_INC	one cycle per response byte (two consecutive for ALU results)	FIFO views: 0x4E then 0x01

5.3 Functional coverage (measured)

A monitor on SYS_CTRL (state, transition, decoded command), the ALU (function at each enable), and the register file (address of each write/read) was attached to the supplied testbench in three runs: default even parity, parity off, and odd parity, each across prescale 32/16/8. (The coverage below comes from the testbench source in +QUICK mode, which issues one vector per ALU function; the transcript in 5.1 is a longer sweep.)

Coverage item	Bins	Hit	Result
SYS_CTRL states	10	10	100%
SYS_CTRL transitions (command paths)	13	13	100%
Commands decoded (AA, BB, CC, DD)	4	4	100%
ALU functions (0x0-0xE)	15	11	73%
Registers written (addresses 0x0-0xF)	16	8	50%
Registers read (addresses 0x0-0xF)	16	6	38%
Prescale settings (32, 16, 8) × parity (even, off, odd)	9	9	100%

ALU function hits (all runs): 0x0: 18, 0x1: 18, 0x2: 27, 0x3: 27, 0x4: 18, 0x5: 9, 0x6: 0, 0x7: 0, 0x8: 9, 0x9: 0, 0xA: 18, 0xB: 18, 0xC: 0, 0xD: 9, 0xE: 18, 0xF: 0.

Coverage gaps (candidates for additional tests): ALU functions not issued in the +QUICK run: NAND (0x6), NOR (0x7), XNOR (0x9), A<B (0xC); unknown command bytes (the default branch of fetch is never taken); FIFO-full back-pressure (FIFO_FULL high for 0 cycles); registers 0x5, 0x6, 0x8-0xA, 0xC-0xE never accessed; parity and stop-bit error frames; a glitch on the start bit; reconfiguration while a frame is in flight. The full (non-QUICK) random mode was not run for coverage.

5.4 Command walkthroughs

Each command is shown as a control-path view (UART receiver, Data Synchronizer, SYS_CTRL state, register-file strobes, FIFO write and TX) followed by two block-level views.

Register write (AA)

AA 03 20 writes 0x20 to address 3 (REG3). Three RX_D_VALID pulses; the synchronizer presents aa, 03, 20; SYS_CTRL steps 0 → 3 → 2 → 0 and the register file gets a WrEn strobe with address 3. No response is produced (TX_OUT stays high). The value equals the reset default, so the array does not visibly change.

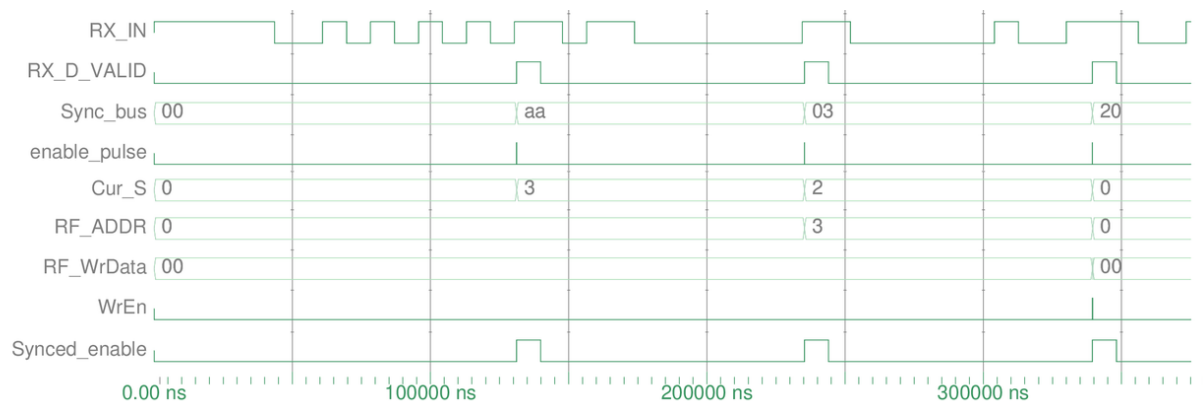


Figure 7. Write, control path (0 – 0.39 ms).

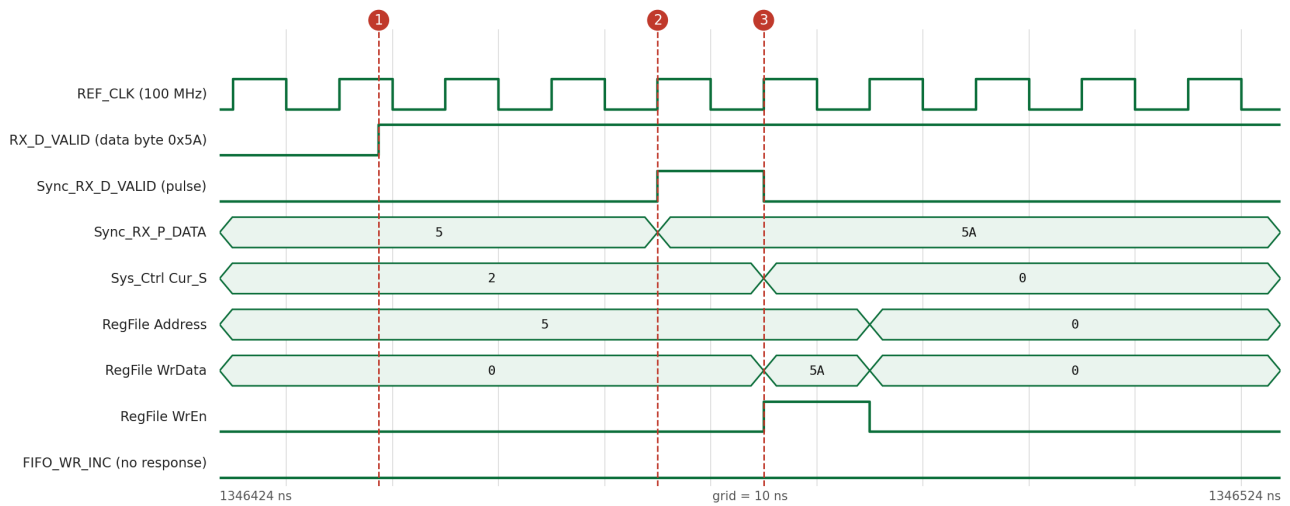


Figure 8. Write zoom (100 ns): the data byte 0x5A of AA 05 5A crosses the synchronizer (marker 2) and SYS_CTRL strobes WrEn about 36 ns after RX_D_VALID rose (marker 3); FIFO_WR_INC stays low because a write has no response.

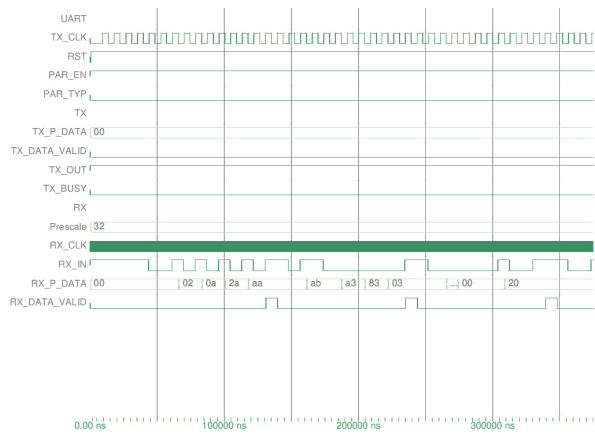


Figure 9. UART: frames decoded into RX_P_DATA.

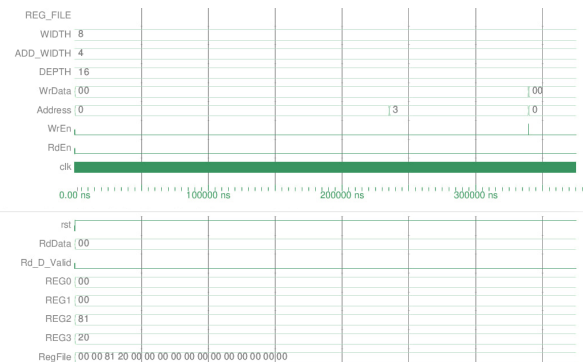


Figure 10. Reg_File: address and WrEn strobe.

Register read (BB)

BB 03 returns **0x20**. SYS_CTRL goes 0 → 1 (RF_Read) → F → 0: RdEn with address 3, RdData = 0x20 with Rd_D_Valid, then the byte is pushed to the FIFO. EMPTY falls ~17 μs after the write (read-domain synchronizer) and R_INC pops the entry once TX_BUSY rises.

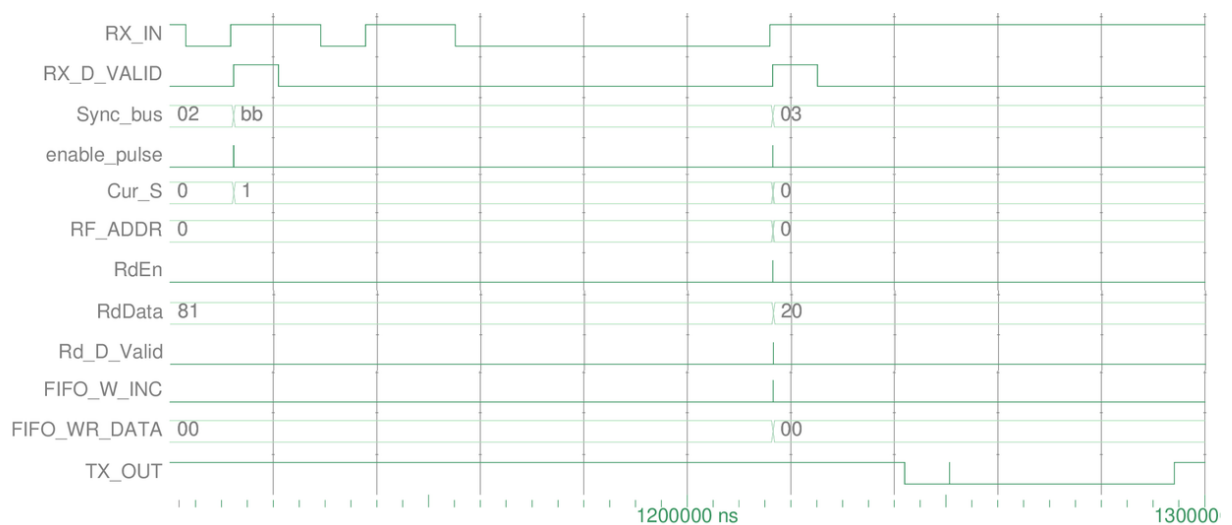


Figure 11. Read, control path (1.20 - 1.30 ms).

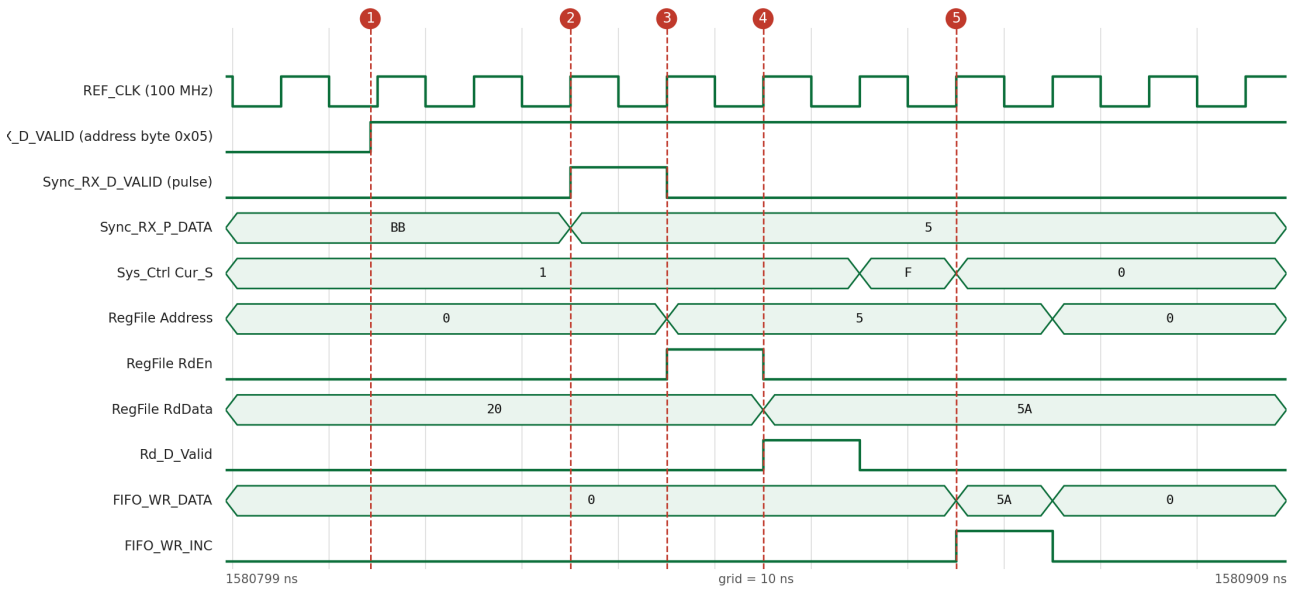


Figure 12. Read zoom (110 ns): address byte of BB 05 (reads back the 0x5A just written). 1: RX_D_VALID; 2: pulse; 3: RdEn; 4: Rd_D_Valid with RdData; 5: FIFO_WR_INC with the byte.

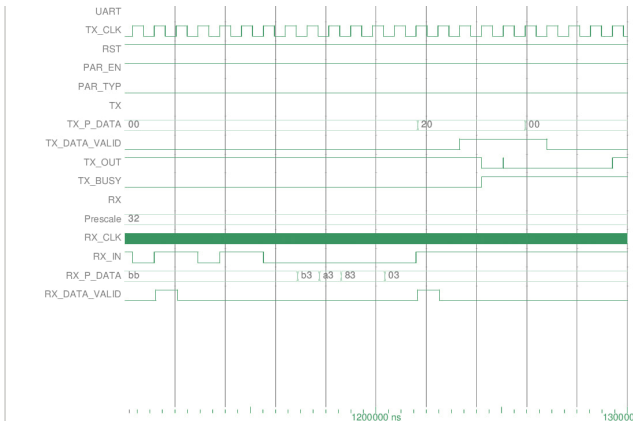


Figure 13. UART: TX_P_DATA = 0x20 and the TX frame.

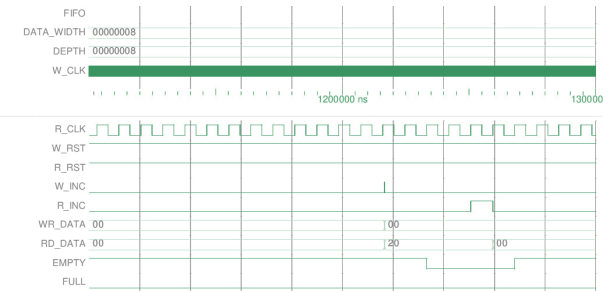


Figure 14. ASYNC_FIFO: write, EMPTY fall, R_INC pop.

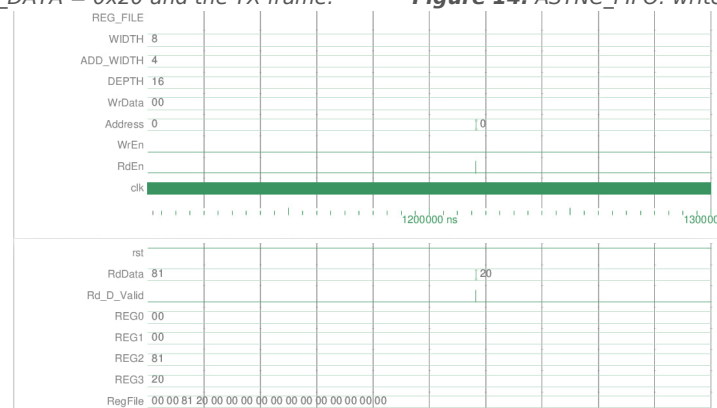


Figure 15. Reg_File: RdEn pulse, address 3, RdData = 0x20 with Rd_D_Valid.

ALU with operands (CC)

CC 62 EC 00 = ADD, A = 0x62, B = 0xEC. SYS_CTRL: 4 (REG0 $\leftarrow$ 0x62) $\rightarrow$ 5 (REG1 $\leftarrow$ 0xEC) $\rightarrow$ 7 (ALU_FUNC) $\rightarrow$ return to 0. Result 0x62 + 0xEC = 0x014E is sent **0x4E then 0x01**; one gated ALU clock pulse computes it (Section 5.6).

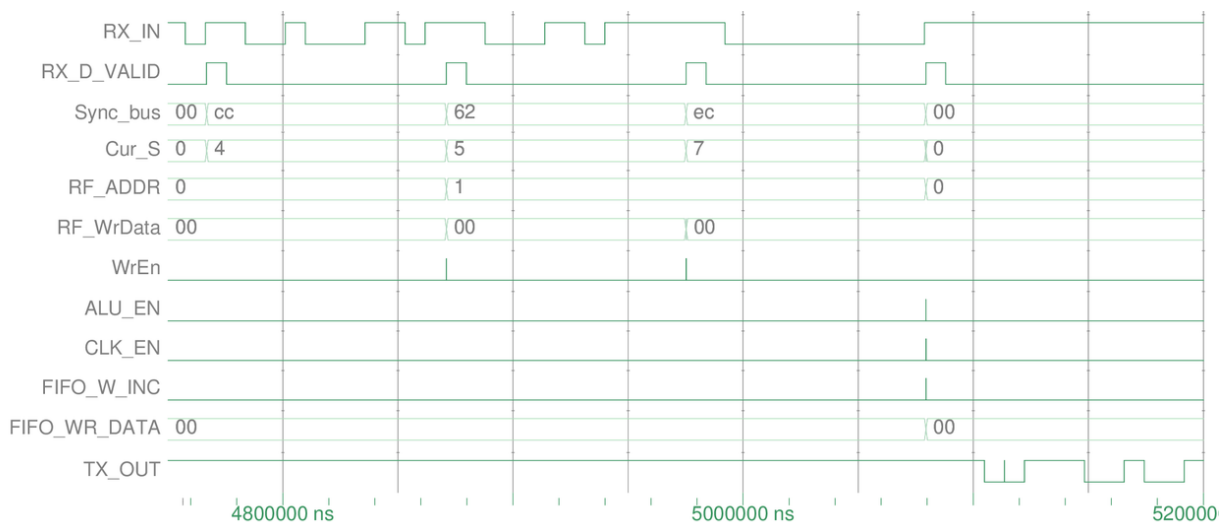


Figure 16. ALU with operands, control path (4.8 – 5.2 ms).

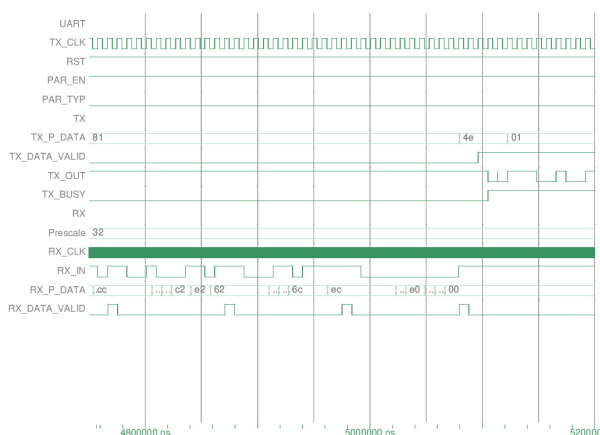


Figure 17. UART: RX CC 62 EC 00; TX 0x4E, 0x01.

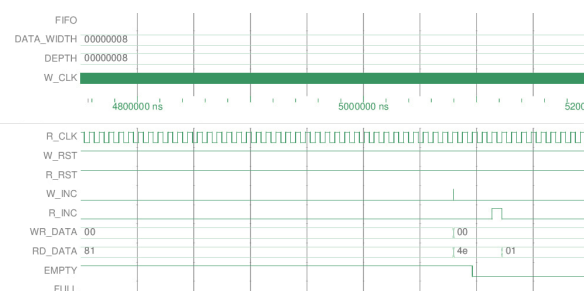


Figure 18. ASYNC_FIFO: two writes, reads 0x4E and 0x01.

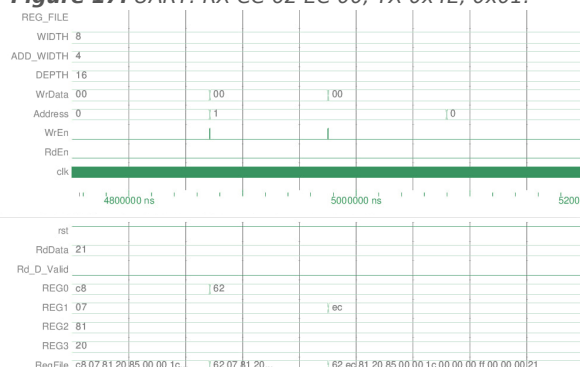


Figure 19. Reg_File: REG0 = 0x62, REG1 = 0xEC loaded by WrEn strobes.

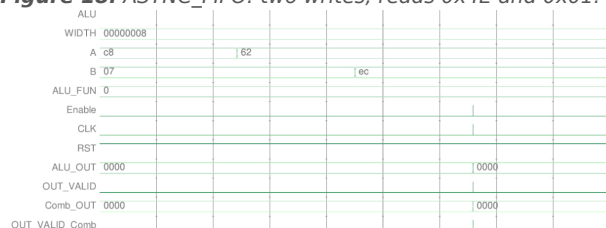


Figure 20. ALU: A, B updated; Enable and clock pulse at the end.

ALU without operands (DD)

DD 00 = ADD on the existing REG0 = 0x7A and REG1 = 0x0C; response **0x86, 0x00**. Path 0 → 7 → 0 is shorter: only two frames (191 μ s) instead of four. The ALU view shows A = 0x7A, B = 0x0C and the single Enable pulse.

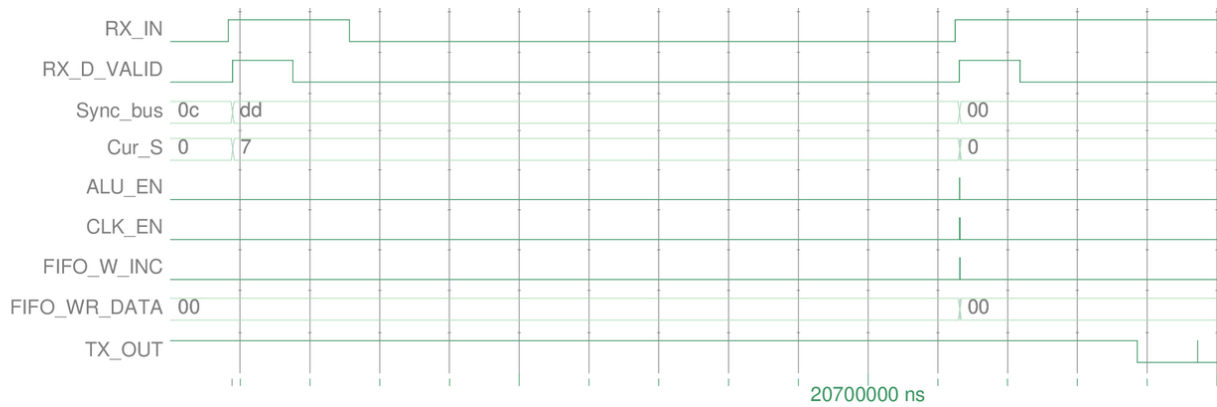


Figure 21. ALU without operands, control path (~20.7 ms).

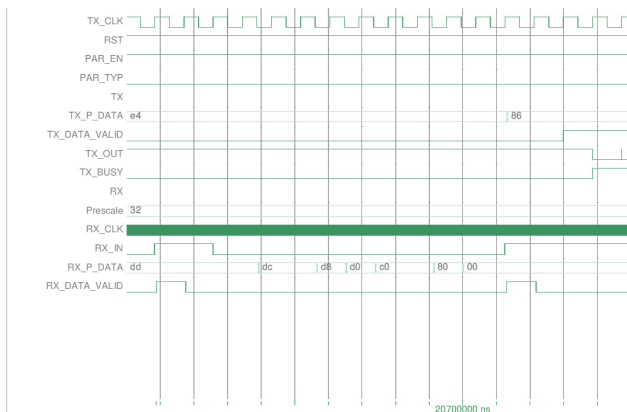


Figure 22. UART: RX DD 00; TX 0x86.



Figure 23. ALU: A = 0x7A, B = 0x0C, Enable pulse.

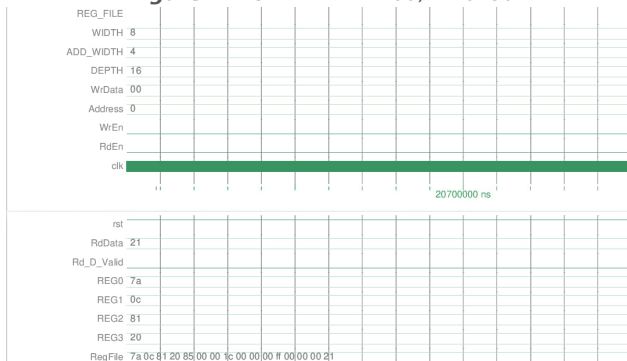


Figure 24. Reg_File: REG0 = 0x7A, REG1 = 0x0C (operands).

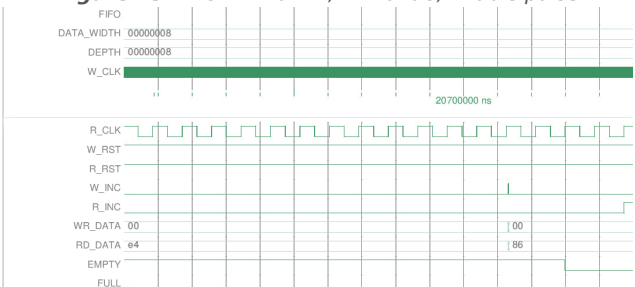


Figure 25. ASYNC_FIFO: result byte 0x86 written and read.

ALU results seen in the supplied log

Selected entries from the sweep (all passed). The log labels every operation "ADD"; decoding the returned values shows the function actually exercised.

Function (decoded)	A	B	Result returned (MSB:LSB)	Check
ADD	0x62	0xEC	0x014E	$A + B = 0x14E$
SUB	0xC7	0x08	0x00BF	$A - B = 0xBF$
SUB	0x98	0x45	0x0053	$A - B = 0x53$
MUL	0xC6	0x09	0x06F6	$A \times B = 0x6F6$
MUL	0xC5	0xE8	0xB288	$A \times B = 0xB288$
MUL (corner)	0xFF	0xFF	0xFE01	$A \times B = 0xFE01$
DIV	0xC5	0x0A	0x0013	$A \div B = 0x13$
AND	0x8F	0x17	0x0007	$A \& B = 0x07$
OR	0x5E	0x94	0x00DE	$A B = 0xDE$
XOR	0x16	0x15	0x0003	$A \wedge B = 0x03$

Function (decoded)	A	B	Result returned (MSB:LSB)	Check
A > B	0xC0	0x0F	0x0002	encoded 2
A $\gg$ 1	0xBF	0x10	0x005F	A $\gg$ 1 = 0x5F
A $\ll$ 1	0xBE	0x11	0x017C	A $\ll$ 1 = 0x17C
DD: ADD	0x7A	0x0C	0x0086	A + B = 0x86
DD: SUB	0x7A	0x0C	0x006E	A – B = 0x6E
DD: SHL	0x7A	0x0C	0x00F4	A $\ll$ 1 = 0xF4

Timing checks per prescale

Prescale	REG2 read-back	RX_CLK period	TX_CLK period
32	0x81	271.3 ns (UART_CLK $\div$ 1)	8680.6 ns (1 bit @ 115 200)
16	0x41	542.5 ns ($\div$ 2)	8680.6 ns
8	0x21	1085.1 ns ($\div$ 4)	8680.6 ns

5.5 Configuration matrix

Parity setting	Prescale 32	Prescale 16	Prescale 8	Checks passed / failed
Even parity (default)	pass	pass	pass	159 / 0
Parity off	pass	pass	pass	159 / 0
Odd parity	pass	pass	pass	159 / 0

The matrix was run from the supplied testbench source (+QUICK: one vector per ALU function) at REF_CLK = 100 MHz; every configuration passes with zero failures, so the receiver, transmitter and SYS_CTRL behave identically across prescale and parity settings.

5.6 Multi-clock behaviour in simulation

The zoom below (150 ns, generated from the RTL simulation) follows the function byte of a CC command through both clock-domain crossings and the clock gate.

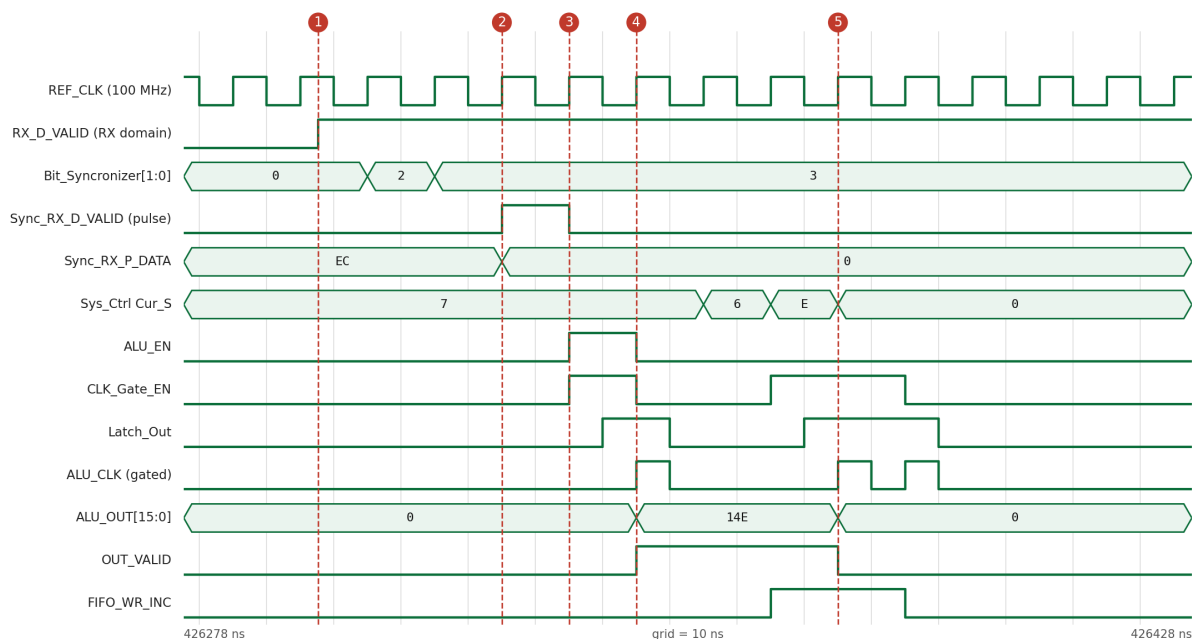


Figure 26. Zoomed simulation of one ALU command. 1: RX_D_VALID rises; 2: synchronized pulse and captured byte; 3: ALU_EN and CLK_EN registered; 4: single compute edge on ALU_CLK; 5: first flush edge.

Marker	Time	Event
1	0 ns	UART_RX raises RX_D_VALID (a level lasting the stop bit); the synchronized bus still holds the previous byte (0xEC).
2	+27 ns	Sync_RX_D_VALID pulses for one REF_CLK cycle and the new byte (0x00 = ADD) is captured.
3	+37 ns	SYS_CTRL (state 7, ALU_FUNC) registers ALU_EN and CLK_Gate_EN; the latch opens in the low phase (Latch_Out, +42 ns).
4	+47 ns	One whole ALU_CLK pulse: ALU_OUT = 0x14E (0x62 + 0xEC), OUT_VALID = 1. States 6 and E queue 0x4E and 0x01 (FIFO_WR_INC high for two cycles, +67 ns).
5	+77 ns	Flush edge (ALU_EN low): ALU_OUT and OUT_VALID return to 0; the high byte is sampled on this same edge.

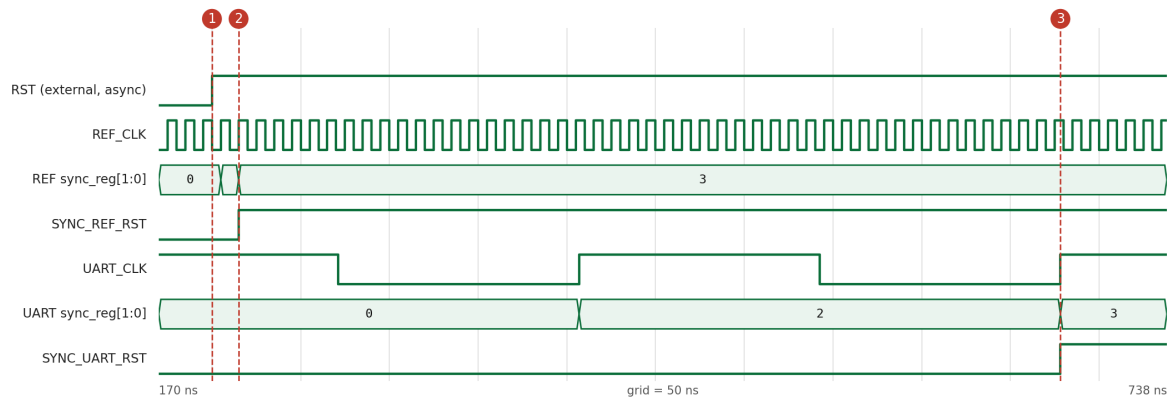


Figure 27. Reset release: (1) RST released at 200 ns; (2) SYNC_REF_RST releases 15 ns later (two REF_CLK edges); (3) SYNC_UART_RST releases at 678 ns (two UART_CLK edges).

FIFO pointer crossing

The FIFO write pointer is gray-coded and registered in REF_CLK, then crosses into TX_CLK through a 2-FF synchronizer. The figure follows one write; the read-side EMPTY flag drops only on the second TX_CLK edge after the write, which is the 17 μ s latency of Section 3.2.

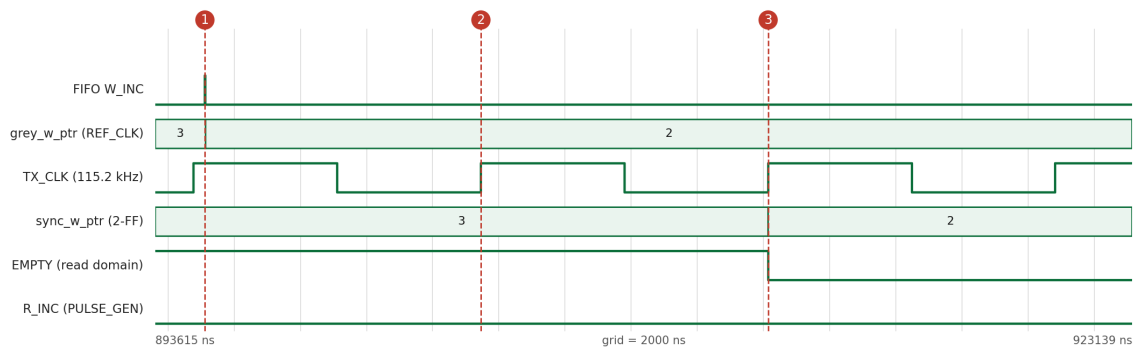


Figure 28. FIFO crossing (the response byte of BB 03). 1: W_INC writes the entry and grey_w_ptr changes by a single bit (0011 → 0010); 2: first TX_CLK edge captures it in the first synchronizer flop; 3: second TX_CLK edge presents it as sync_w_ptr and EMPTY falls, 17.0 μ s after the write.

UART_RX oversampling

At prescale 32 each bit is 32 RX_CLK cycles. Edge_Count runs 0–31 inside each bit; Sampled_flag fires near the middle of the bit and the majority of the three samples is taken as the bit value. The figure shows the start bit followed by the first two data bits of 0xBB (D0 = 1, D1 = 1).

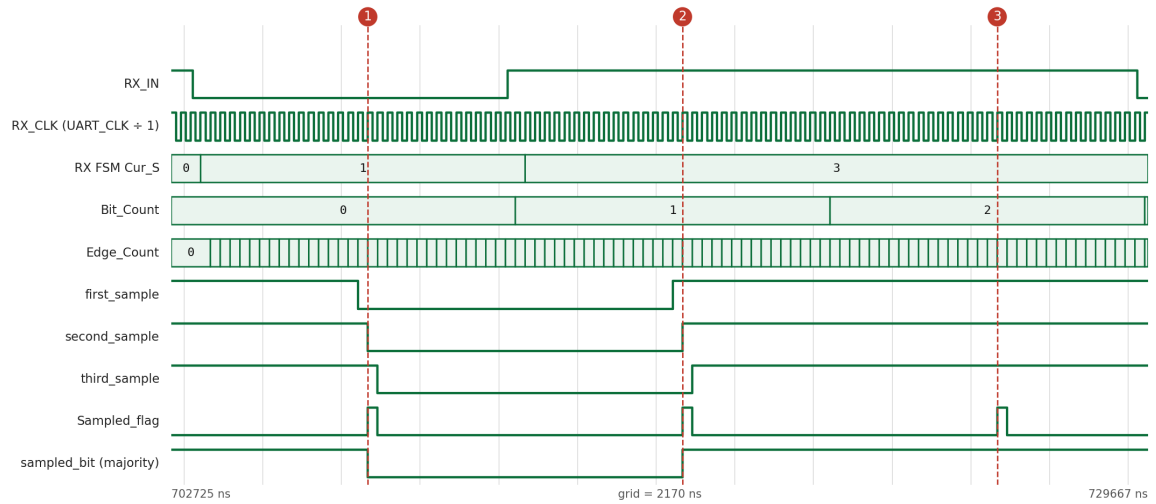


Figure 29. UART_RX oversampling at prescale 32. 1: start-bit centre (all three samples 0, majority 0); 2: D0 centre (majority 1); 3: D1 centre. Bit_Count advances at each bit boundary and the FSM moves from START (1) to DATA (3).

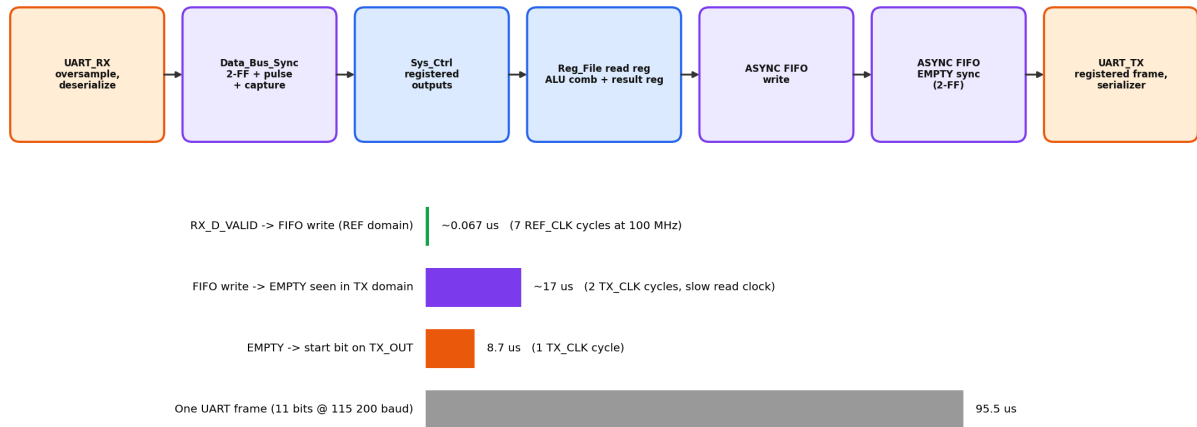


Figure 30. Command-to-response pipeline with measured latencies, drawn to scale against one UART frame (95.5 μ s).

Measured quantity (REF_CLK = 100 MHz)	Value
RX_D_VALID $\rightarrow$ synchronized pulse	21-28 ns (2-3 cycles; varies with phase, as expected for an asynchronous crossing)
Register write: RX_D_VALID (data byte) $\rightarrow$ WrEn	36 ns
Register read: RX_D_VALID (address byte) $\rightarrow$ RdEn / Rd_D_Valid / FIFO_WR_INC	31 / 41 / 61 ns
ALU (DD): RX_D_VALID (function byte) $\rightarrow$ ALU_EN / FIFO_WR_INC	38 / 68 ns
ALU clock edges per command / REF_CLK edges in the same 677 μ s	3 / 67 709
FIFO write $\rightarrow$ EMPTY de-asserts in TX domain	17.0 μ s (2 TX_CLK cycles)
EMPTY de-assert $\rightarrow$ start bit on TX_OUT	8.7 μ s (1 TX_CLK cycle); read response starts 25.8 μ s after the address byte
REF / UART reset release after RST rises	15 ns / 478 ns

5.7 Verification traceability

How each specification item was verified in simulation, and what remains open.

Specification item	How it was verified	Result
Command set AA / BB / CC / DD	Directed sweep; responses decoded and compared (Section 5.1); all four commands decoded by SYS_CTRL (5.3)	pass

Specification item	How it was verified	Result
Register file reset values and read/write (REG2 = 0x81, REG3 = 0x20, 0x4-0xF)	Read-back of REG2/REG3 and four general registers at every prescale	pass
ALU functions (15)	Supplied sweep (11 distinct functions decoded in 5.4); +QUICK run hits 11 of 15 function codes	pass; gaps listed in 5.3
Prescale 32 / 16 / 8 and parity even / off / odd	Configuration matrix, 9 combinations (5.5)	pass (0 failures)
115 200 baud TX, derived RX clock	TX_CLK period 8680.6 ns; RX_CLK 271.3 / 542.5 / 1085.1 ns	pass
CDC: RX → REF_CLK and REF_CLK → TX_CLK	Zoomed waveforms (5.6): synchronized pulse, gray pointer crossing, 17.0 μ s EMPTY latency; every response byte arrives intact and in order	pass
RDC: per-domain reset release	Reset-release waveform: 15 ns vs 478 ns; no spurious RX_D_VALID during the skew	pass
Clock gating	3 ALU_CLK edges per command against 67 709 REF_CLK edges; correct results through the gated clock	pass
Pipeline latency	RX_D_VALID → first FIFO write 61-68 ns; throughput limited by the serial link	measured
Error handling (parity, stop bit, start glitch), unknown commands, FIFO full	not exercised by the supplied tests	open

5.8 Reproducing the simulations

Sources: the RTL archive with the .sv versions of UART_RX_FSM and UART_TX_FSM, and System_Top_tb.sv with the reference-clock half-period set to 5.0 ns (100 MHz); UART_CLK is 3.6864 MHz. Run-time options: +QUICK (one vector per ALU function), +PAR_EN=<0|1>, +PAR_TYP=<0|1>. The testbench prints one PASS/FAIL line per check and a final PASSED/FAILED summary. The zoomed waveforms in 5.4 and 5.6 were produced from a short directed run (CC, BB, AA and DD commands) with full signal dump.

6. Backend summary (brief)

Synthesis, DFT and verification results from the supplied reports. The full backend flow will be added later.

Item	Result
Area	22 405.5 (library units, as reported)
Timing	WNS 0.00, TNS 0.0 (met)
Power (total)	0.450 (internal 94.4 %, switching 2.6 %, leakage 3.0 %; units as reported)
Stuck-at test coverage	99.40 % = 15 350 / (15 512 – 69 undetectable); 79 ATPG-untestable, 14 not detected
Formal verification	"Verification Succeeded"

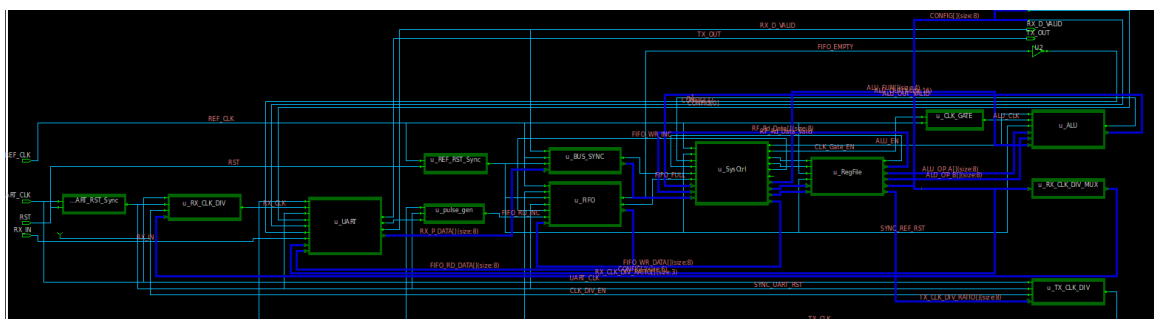


Figure 31. Post-synthesis schematic of System_Top.

Note: the synthesized hierarchy shows a single `u_UART` wrapper, whereas `System_Top_pre_dft.v` instantiates `UART_RX` and `UART_TX` directly; confirm the archived RTL is the synthesized snapshot.

7. Review notes

#	Observation	Action
1	The archive has both .v and .sv versions of UART_RX_FSM and UART_TX_FSM with the same module names; the .v RX FSM is older and corrupts the next frame whenever the parity bit is 0.	Compile only the .sv FSMs.
2	REG2/REG3 cross into the UART family without a synchronizer; RX_IN has no separate 2-FF stage.	Declare static in constraints; add a 2-FF on RX_IN.
3	Coverage gaps (Section 5.3): ALU functions NAND, NOR, XNOR and A<B are not issued in +QUICK mode; unknown commands, FIFO-full stalls and parity/stop-bit error frames are not tested.	Add directed tests for these bins.
4	Spec quotes general registers as 0x4-0x15; with a 4-bit address the range is 0x4-0xF.	Correct the spec text.
5	ALU_CLK must not lead REF_CLK after CTS (high result byte is sampled on the flush edge).	Balance the gated clock.